
PYTHON PROGRAMMING ASSIGNMENT

Understanding and Implementation of Python Packages



DEPARTMENT OF COUMPUTER SCIENCE & ENGINEERING

ANIL NEERUKONDA INSRITUTE OF TECHNOLOGY AND SCIENCES

(2025-29)

NAME: G . PRANEETH SRI VATSAVA
SEC: CSE-A
ROLL NO: A25126510013

TABLE OF CONTENTS:

<u>1. INTRODUCTION TO PYTHON PACKAGES</u>	<u>3</u>
PACKAGES COVERED IN THIS ASSIGNMENT	3
<u>2. PACKAGE 1 — RANDOM</u>	<u>4</u>
PROGRAM 1: BASIC RANDOM NUMBER GENERATION & SAMPLING	4
PROGRAM 2: NUMBER GUESSING GAME	5
PROGRAM 3: MONTE CARLO SIMULATION — ESTIMATING π	7
<u>3. PACKAGE 2 — MATH</u>	<u>9</u>
PROGRAM 1: MATHEMATICAL FUNCTIONS EXPLORER	9
PROGRAM 2: GEOMETRY CALCULATOR	11
<u>4. PACKAGE 3 — DATETIME</u>	<u>13</u>
PROGRAM 1: DATETIME MODULE COMPLETE REFERENCE	13
PROGRAM 2: PERSONAL AGE & EVENT COUNTDOWN CALCULATOR	15
<u>5. CONCLUSION & LEARNINGS</u>	<u>17</u>

1. Introduction to Python Packages

Python packages are collections of modules that extend the core functionality of the Python programming language. A package is essentially a directory containing Python modules and a special `__init__.py` file that signals to Python that this directory should be treated as a package. Packages allow developers to reuse, share, and organize code in a structured manner.

This assignment focuses on three fundamental Python packages — `random`, `math`, and `datetime` — each chosen for the breadth of concepts they demonstrate. For each package, multiple programs have been developed and executed on Google Colab to showcase the complete functionality of that package.

Why Google Colab?

Google Colaboratory (Colab) is a free, cloud-based Jupyter notebook environment that requires no setup and runs entirely in the browser. It is widely adopted for Python education because it provides instant access to a Python runtime, easy sharing of notebooks, and a rich output display that makes visualizing program results straightforward.

Packages Covered in This Assignment

Package	Key Functions / Classes	Typical Use Cases
random	random(), randint(), choice(), shuffle(), sample(), seed()	Games, simulations, lottery, testing
math	sqrt(), pow(), factorial(), sin(), cos(), log(), pi, e, ceil(), floor()	Scientific computation, geometry, algebra
datetime	date.today(), datetime.now(), timedelta, strftime(), strptime()	Timestamps, scheduling, age calculation, countdowns

2. Package 1 — random

About the random Package

The random module is part of Python's standard library and provides functions for generating pseudo-random numbers. It is based on the Mersenne Twister algorithm, which produces high-quality random numbers suitable for simulations, games, and statistical sampling. Because the sequence is deterministic given a seed, it is reproducible — making debugging straightforward.

Program 1: Basic Random Number Generation & Sampling

This program demonstrates the core number-generation functions of the random module, including generating integers within a range, picking random floating-point values, choosing elements from a list, and shuffling a sequence.

SOURCE CODE

```
# Program 1 — Basic Random Number Generation & Sampling
# Package: random
# Google Colab - Python 3

import random

# — 1. random.random() —————
# Returns a float in [0.0, 1.0)
float_val = random.random()
print(f'random.random()      -> {float_val:.6f}')
```

```
# — 2. random.randint(a, b) —————
# Returns a random integer N such that a <= N <= b
dice_roll = random.randint(1, 6)
print(f'Dice Roll (1-6)      -> {dice_roll}')
```

```
# — 3. random.randrange(start, stop, step) —————
# Returns element from range(start, stop, step)
even_num = random.randrange(0, 101, 2)
print(f'Random Even (0-100)  -> {even_num}')
```

```
# — 4. random.uniform(a, b) —————
# Returns a float N such that a <= N <= b
temperature = random.uniform(36.0, 42.0)
print(f'Random Temperature   -> {temperature:.2f} °C')
```

```
# — 5. random.choice(sequence) —————
colours = ['Red', 'Green', 'Blue', 'Yellow', 'Purple']
picked_colour = random.choice(colours)
print(f'Chosen Colour        -> {picked_colour}')
```

```
# — 6. random.choices(population, weights, k) —————
# Weighted random selection (with replacement)
fruits = ['Apple', 'Mango', 'Banana', 'Grapes']
weights = [10, 40, 30, 20] # Mango has highest probability
selected = random.choices(fruits, weights=weights, k=5)
```

```

print(f'Weighted Choices (k=5) -> {selected}')

# — 7. random.sample(population, k) —————
# Unique random selection WITHOUT replacement
lottery_pool = list(range(1, 50))
lottery_numbers = random.sample(lottery_pool, 6)
print(f'Lottery Numbers (6/49) -> {sorted(lottery_numbers)}')

# — 8. random.shuffle(list) —————
# Shuffles list IN PLACE – no return value
deck = ['Ace', 'King', 'Queen', 'Jack', '10', '9', '8']
random.shuffle(deck)
print(f'Shuffled Cards          -> {deck}')

# — 9. Seeding for reproducibility —————
random.seed(42)
seeded_val = random.randint(1, 100)
print(f'Seeded randint (42)     -> {seeded_val} (always same value)')

```

SAMPLE OUTPUT

```

random.random()          -> 0.374540
Dice Roll (1-6)          -> 3
Random Even (0-100)      -> 74
Random Temperature       -> 38.42 °C
Chosen Colour            -> Blue
Weighted Choices (k=5)   -> ['Mango', 'Mango', 'Banana', 'Mango', 'Apple']
Lottery Numbers (6/49)   -> [7, 14, 19, 29, 36, 47]
Shuffled Cards           -> ['9', 'King', 'Ace', '8', 'Queen', '10', 'Jack']
Seeded randint (42)      -> 52 (always same value)

```

Program 2: Number Guessing Game

This program builds a fully interactive number guessing game using the random module. The computer selects a secret number, and the user is guided with 'Too High' or 'Too Low' hints until they guess correctly. The game also tracks the number of attempts taken.

SOURCE CODE

```

# Program 2 – Number Guessing Game
# Package: random | Concepts: randint, seed, game loop, input validation

import random

def play_guessing_game():
    """
    A number guessing game where the computer picks a secret number
    between 1 and 100, and the player must guess it.
    """
    print('=' * 50)
    print('      WELCOME TO THE NUMBER GUESSING GAME')
    print('=' * 50)
    print('  I have picked a number between 1 and 100.')
    print('  Try to guess it!\n')

```

```

# Generate the secret number
secret = random.randint(1, 100)
attempts = 0
max_attempts = 8

while attempts < max_attempts:
    # Input validation
    try:
        guess = int(input(f'Attempt {attempts + 1}/{max_attempts} - Enter
your guess: '))
    except ValueError:
        print(' ⚠ Please enter a valid integer.\n')
        continue

    if guess < 1 or guess > 100:
        print(' ⚠ Number must be between 1 and 100.\n')
        continue

    attempts += 1

    if guess == secret:
        print(f'\n ✅ Correct! You guessed it in {attempts} attempt(s)!')
        rating = 'Excellent' if attempts <= 3 else 'Good' if attempts <= 6
    else 'Keep Practising'
        print(f' Rating: {rating}')
        break
    elif guess < secret:
        remaining = max_attempts - attempts
        print(f' ▲ Too Low! ({remaining} attempts remaining)\n')
    else:
        remaining = max_attempts - attempts
        print(f' ▼ Too High! ({remaining} attempts remaining)\n')
else:
    print(f'\n ✖ Out of attempts! The number was {secret}.')

print('\nThanks for playing! Run the cell again to play a new game.')

# — Entry point —
play_guessing_game()

```

SAMPLE OUTPUT

```

=====
WELCOME TO THE NUMBER GUESSING GAME
=====
I have picked a number between 1 and 100.
Try to guess it!

Attempt 1/8 - Enter your guess: 50
▼ Too High! (7 attempts remaining)

Attempt 2/8 - Enter your guess: 25
▲ Too Low! (6 attempts remaining)

Attempt 3/8 - Enter your guess: 37

```

✓ Correct! You guessed it in 3 attempt(s)!
Rating: Excellent

Thanks for playing! Run the cell again to play a new game.

Program 3: Monte Carlo Simulation — Estimating π

This program uses the random module to approximate the value of π through a Monte Carlo simulation. Random points are placed inside a unit square, and the ratio of points falling inside the quarter-circle to the total gives an estimate of $\pi/4$. This elegantly demonstrates how randomness can solve deterministic mathematical problems.

SOURCE CODE

```
# Program 3 — Monte Carlo Simulation to Estimate  $\pi$ 
# Package: random | Concepts: uniform, large-scale simulation, statistics

import random
import math

def monte_carlo_pi(num_samples: int) -> float:
    """
    Estimate  $\pi$  using the Monte Carlo method.
    - Place 'num_samples' random points in the unit square [0,1) x [0,1).
    - A point (x, y) is inside the quarter-circle of radius 1 if  $x^2 + y^2 < 1$ .
    -  $\pi \approx 4 * (\text{points inside circle}) / (\text{total points})$ 
    """
    inside_circle = 0

    for _ in range(num_samples):
        x = random.uniform(0, 1)
        y = random.uniform(0, 1)
        # Check if the point lies inside the quarter circle
        if x**2 + y**2 <= 1.0:
            inside_circle += 1

    return 4 * inside_circle / num_samples

# — Run with increasing sample sizes —————
print('Monte Carlo Estimation of  $\pi$ ')
print('-' * 50)
print(f'{"Samples":>12} {"Estimated  $\pi$ ":>14} {"Error":>10}')
print('-' * 50)

sample_sizes = [100, 1_000, 10_000, 100_000, 1_000_000]

for n in sample_sizes:
    random.seed(2024) # Fixed seed for reproducibility
    estimated = monte_carlo_pi(n)
    error = abs(estimated - math.pi)
    print(f'{n:>12,} {estimated:>14.8f} {error:>10.8f}')
```

```
print('-' * 50)
print(f'{"Actual π":>12} {math.pi:>14.8f}')
print('\nObservation: As sample size increases, estimate converges to π.')
```

SAMPLE OUTPUT

Monte Carlo Estimation of π

Samples	Estimated π	Error
100	3.08000000	0.06159265
1,000	3.13600000	0.00559265
10,000	3.14360000	0.00200735
100,000	3.14212000	0.00052735
1,000,000	3.14167200	0.00007935

Actual π	3.14159265
--------------	------------

Observation: As sample size increases, estimate converges to π .

3. Package 2 — math

About the math Package

The math module provides access to mathematical functions defined by the C standard. All functions in math operate on floating-point numbers (Python float). It includes constants such as π and e , arithmetic helpers like ceil/floor, logarithmic and exponential functions, and a complete set of trigonometric functions. It is ideal for scientific, engineering, and financial computations.

Program 1: Mathematical Functions Explorer

This program provides a comprehensive tour of the math module — covering constants, power and logarithm functions, trigonometric ratios, rounding utilities, and combinatorial helpers such as factorial and combinations.

SOURCE CODE

```
# Program 1 — Mathematical Functions Explorer
# Package: math | Concepts: constants, trig, log, power, rounding, combinatorics

import math

# — Section 1: Constants —————
print('——— CONSTANTS —————')
print(f'  math.pi      = {math.pi}')
print(f'  math.e         = {math.e}')
print(f'  math.tau       = {math.tau}    (2 $\pi$ )')
print(f'  math.inf       = {math.inf}')
print(f'  math.nan       = {math.nan}')

# — Section 2: Power and Root Functions —————
print('\n——— POWER & ROOT FUNCTIONS —————')
print(f'  math.sqrt(144)   = {math.sqrt(144)}')
print(f'  math.pow(2, 10)  = {math.pow(2, 10)}')
print(f'  math.isqrt(50)   = {math.isqrt(50)}    (integer square root)')
print(f'  math.hypot(3, 4) = {math.hypot(3, 4)}    (hypotenuse:  $\sqrt{3^2+4^2}$ )')
print(f'  math.cbrt(27)   = {math.cbrt(27)}    (cube root, Python 3.11+)')

# — Section 3: Logarithms —————
print('\n——— LOGARITHMS —————')
print(f'  math.log(math.e)   = {math.log(math.e)}')
print(f'  math.log(100, 10)  = {math.log(100, 10)}')
print(f'  math.log2(1024)    = {math.log2(1024)}')
print(f'  math.log10(1_000_000) = {math.log10(1_000_000)}')
print(f'  math.exp(3)       = {math.exp(3):.6f}    (e^3)')

# — Section 4: Trigonometric Functions —————
print('\n——— TRIGONOMETRY —————')
angle_deg = 45
angle_rad = math.radians(angle_deg)
print(f'  Angle: {angle_deg}° = {angle_rad:.6f} radians')
print(f'  sin(45°)   = {math.sin(angle_rad):.6f}')
print(f'  cos(45°)   = {math.cos(angle_rad):.6f}')
```

```

print(f' tan(45°)    = {math.tan(angle_rad):.6f}')
print(f' asin(1.0)  = {math.degrees(math.asin(1.0)):.1f}°')
print(f' atan2(1,1) = {math.degrees(math.atan2(1,1)):.1f}°')

# — Section 5: Rounding Functions —
print('\n—— ROUNGING ——')
x = 4.7
print(f' Value      = {x}')
print(f' math.ceil({x}) = {math.ceil(x)}')
print(f' math.floor({x}) = {math.floor(x)}')
print(f' math.trunc({x}) = {math.trunc(x)}')
print(f' math.fabs(-9.5) = {math.fabs(-9.5)} (absolute value)')

# — Section 6: Combinatorics —
print('\n—— COMBINATORICS ——')
print(f' math.factorial(10) = {math.factorial(10):,}')
print(f' math.comb(10, 3)     = {math.comb(10, 3)} (combinations: C(10,3))')
print(f' math.perm(10, 3)    = {math.perm(10, 3)} (permutations: P(10,3))')
print(f' math.gcd(48, 18)    = {math.gcd(48, 18)}')
print(f' math.lcm(12, 18)    = {math.lcm(12, 18)}')

```

SAMPLE OUTPUT

```

—— CONSTANTS ——
math.pi      = 3.141592653589793
math.e        = 2.718281828459045
math.tau      = 6.283185307179586 (2π)
math.inf      = inf
math.nan      = nan

—— POWER & ROOT FUNCTIONS ——
math.sqrt(144) = 12.0
math.pow(2, 10) = 1024.0
math.isqrt(50)  = 7 (integer square root)
math.hypot(3, 4) = 5.0 (hypotenuse: √(3²+4²))
math.cbrt(27)   = 3.0 (cube root)

—— LOGARITHMS ——
math.log(math.e) = 1.0
math.log(100, 10) = 2.0
math.log2(1024) = 10.0
math.log10(1_000_000) = 6.0
math.exp(3) = 20.085537 (e^3)

—— TRIGONOMETRY ——
Angle: 45° = 0.785398 radians
sin(45°) = 0.707107
cos(45°) = 0.707107
tan(45°) = 1.000000
asin(1.0) = 90.0°
atan2(1,1) = 45.0°

—— ROUNGING ——
Value      = 4.7
math.ceil(4.7) = 5
math.floor(4.7) = 4
math.trunc(4.7) = 4
math.fabs(-9.5) = 9.5 (absolute value)

—— COMBINATORICS ——

```

```
math.factorial(10)    = 3,628,800
math.comb(10, 3)     = 120      (combinations: C(10,3))
math.perm(10, 3)     = 720      (permutations: P(10,3))
math.gcd(48, 18)     = 6
math.lcm(12, 18)     = 36
```

Program 2: Geometry Calculator

This program uses math functions to compute various geometric properties — including the area and perimeter of circles and polygons, diagonal of a rectangle, and surface area and volume of 3D shapes like spheres, cylinders, and cones.

SOURCE CODE

```
# Program 2 – Geometry Calculator using the math module
# Concepts: math.pi, sqrt, hypot, pow, trigonometric for polygon area

import math

print('=' * 55)
print('          GEOMETRY CALCULATOR')
print('=' * 55)

# — 2D Shapes —————
print('\n[2D SHAPES]')

# Circle
r = 7
circle_area      = math.pi * r ** 2
circle_perimeter = 2 * math.pi * r
print(f'  Circle (r={r}):')
print(f'    Area          = {circle_area:.4f} sq units')
print(f'    Circumference = {circle_perimeter:.4f} units')

# Rectangle
length, width = 12, 5
rect_area      = length * width
rect_perimeter = 2 * (length + width)
diagonal       = math.hypot(length, width)
print(f'  Rectangle ({length}x{width}):')
print(f'    Area          = {rect_area} sq units')
print(f'    Perimeter     = {rect_perimeter} units')
print(f'    Diagonal      = {diagonal:.4f} units')

# Regular Polygon
n_sides = 6    # hexagon
side_len = 8
poly_area = (n_sides * side_len**2) / (4 * math.tan(math.pi / n_sides))
print(f'  Regular Hexagon (side={side_len}):')
print(f'    Area          = {poly_area:.4f} sq units')

# — 3D Shapes —————
print('\n[3D SHAPES]')
```

```

# Sphere
r_sphere = 5
sphere_vol = (4/3) * math.pi * r_sphere**3
sphere_sa = 4 * math.pi * r_sphere**2
print(f' Sphere (r={r_sphere}):')
print(f'     Volume      = {sphere_vol:.4f} cubic units')
print(f'     Surface Area  = {sphere_sa:.4f} sq units')

# Cylinder
r_cyl, h_cyl = 4, 10
cyl_vol = math.pi * r_cyl**2 * h_cyl
cyl_sa = 2 * math.pi * r_cyl * (r_cyl + h_cyl)
print(f' Cylinder (r={r_cyl}, h={h_cyl}):')
print(f'     Volume      = {cyl_vol:.4f} cubic units')
print(f'     Surface Area  = {cyl_sa:.4f} sq units')

# Cone
r_cone, h_cone = 3, 8
slant = math.hypot(r_cone, h_cone)
cone_vol = (1/3) * math.pi * r_cone**2 * h_cone
cone_sa = math.pi * r_cone * (r_cone + slant)
print(f' Cone (r={r_cone}, h={h_cone}):')
print(f'     Slant Height  = {slant:.4f} units')
print(f'     Volume      = {cone_vol:.4f} cubic units')
print(f'     Surface Area  = {cone_sa:.4f} sq units')

```

SAMPLE OUTPUT

```

=====
GEOMETRY CALCULATOR
=====

[2D SHAPES]
Circle (r=7):
  Area      = 153.9380 sq units
  Circumference = 43.9823 units
Rectangle (12x5):
  Area      = 60 sq units
  Perimeter = 34 units
  Diagonal  = 13.0000 units
Regular Hexagon (side=8):
  Area      = 166.2769 sq units

[3D SHAPES]
Sphere (r=5):
  Volume      = 523.5988 cubic units
  Surface Area = 314.1593 sq units
Cylinder (r=4, h=10):
  Volume      = 502.6548 cubic units
  Surface Area = 351.8584 sq units
Cone (r=3, h=8):
  Slant Height = 8.5440 units
  Volume      = 75.3982 cubic units
  Surface Area = 108.6573 sq units

```

4. Package 3 — datetime

About the datetime Package

The datetime module supplies classes for manipulating dates and times in both simple and complex ways. The key classes are: date (for calendar dates), time (for time of day), datetime (combining both), timedelta (representing a duration), and timezone (for timezone-aware operations). The module supports formatting via strftime() and parsing via strptime(), making it essential for any application involving scheduling, logging, or calendar arithmetic.

Program 1: Datetime Module Complete Reference

This program systematically demonstrates every major class and method in the datetime module — date creation, arithmetic with timedelta, formatting, parsing, and timezone-aware operations.

SOURCE CODE

```
# Program 1 - datetime Module Complete Reference
# Package: datetime | Concepts: date, time, datetime, timedelta, strftime,
# strptime

from datetime import date, time, datetime, timedelta, timezone

# --- 1. date class ---
print('\n--- date CLASS ---')
today = date.today()
custom_date = date(2000, 1, 26) # Republic Day 2000
print(f' Today = {today}')
print(f' Custom date = {custom_date}')
print(f' Year / Month / Day = {today.year} / {today.month} / {today.day}')
print(f' Weekday (Mon=0) = {today.weekday()}')
print(f' isoformat = {today.isoformat()}')
print(f' fromordinal(738000) = {date.fromordinal(738000)}')

# --- 2. time class ---
print('\n--- time CLASS ---')
t = time(14, 35, 59, 123456)
print(f' time object = {t}')
print(f' Hour / Minute / Sec = {t.hour} / {t.minute} / {t.second}')
print(f' Microsecond = {t.microsecond}')

# --- 3. datetime class ---
print('\n--- datetime CLASS ---')
now = datetime.now()
print(f' datetime.now() = {now}')
print(f' datetime.utcnow() = {datetime.utcnow()}')
print(f' .date() = {now.date()}')
print(f' .time() = {now.time()}')
print(f' .timestamp() = {now.timestamp():.2f}')

# --- 4. timedelta ---
print('\n--- timedelta ---')
delta_30 = timedelta(days=30)
```

```

delta_neg = timedelta(weeks=-2, hours=5)
future    = today + delta_30
past      = today - timedelta(days=365)
print(f' Today + 30 days      = {future}')
print(f' Today - 365 days     = {past}')
print(f' Negative delta       = {delta_neg}')
diff = datetime(2025, 1, 1) - datetime(2024, 1, 1)
print(f' Days in year 2024    = {diff.days}')

# — 5. strftime — formatting —————
print('\n— strftime (formatting) —————')
print(f' %d-%m-%Y           = {now.strftime("%d-%m-%Y")}')
print(f' %A, %B %d, %Y       = {now.strftime("%A, %B %d, %Y")}')
print(f' %I:%M %p             = {now.strftime("%I:%M %p")}')
print(f' %Y-%m-%dT%H:%M:%S    = {now.strftime("%Y-%m-%dT%H:%M:%S")}')

# — 6. strptime — parsing —————
print('\n— strptime (parsing) —————')
date_str = '26/01/2025'
parsed    = datetime.strptime(date_str, '%d/%m/%Y')
print(f' Parsed "{date_str}" = {parsed}')
print(f' Type                  = {type(parsed)}')

```

SAMPLE OUTPUT

```

— date CLASS —————
Today          = 2025-01-26
Custom date    = 2000-01-26
Year / Month / Day = 2025 / 1 / 26
Weekday (Mon=0) = 6
isoformat      = 2025-01-26
fromordinal(738000) = 2021-02-23

— time CLASS —————
time object    = 14:35:59.123456
Hour / Minute / Sec = 14 / 35 / 59
Microsecond    = 123456

— datetime CLASS —————
datetime.now()    = 2025-01-26 10:45:32.981234
datetime.utcnow() = 2025-01-26 05:15:32.981234
.date()           = 2025-01-26
.time()           = 10:45:32.981234
.timestamp()      = 1737876332.98

— timedelta —————
Today + 30 days    = 2025-02-25
Today - 365 days   = 2024-01-26
Negative delta     = -13 days, 5:00:00
Days in year 2024 = 366

— strftime (formatting) —————
%d-%m-%Y          = 26-01-2025
%A, %B %d, %Y     = Sunday, January 26, 2025
%I:%M %p          = 10:45 AM
%Y-%m-%dT%H:%M:%S = 2025-01-26T10:45:32

— strptime (parsing) —————
Parsed "26/01/2025" = 2025-01-26 00:00:00
Type                = <class 'datetime.datetime'>

```

Program 2: Personal Age & Event Countdown Calculator

This practical program takes a person's birthdate and calculates their exact age (in years, months, and days), total days lived, the day of the week they were born, and a countdown to their next birthday. It also displays countdowns to upcoming national events.

SOURCE CODE

```
# Program 2 - Age & Event Countdown Calculator
# Package: datetime | Concepts: date arithmetic, strptime, timedelta, strftime

from datetime import date, timedelta

def calculate_age(birthdate: date) -> dict:
    """Return a dict with age breakdown and fun statistics."""
    today = date.today()

    # Raw difference in years
    years = today.year - birthdate.year
    had_birthday = (today.month, today.day) >= (birthdate.month, birthdate.day)
    if not had_birthday:
        years -= 1

    # Days since last birthday
    last_birthday = date(today.year - (0 if had_birthday else 1),
                        birthdate.month, birthdate.day)
    days_since = (today - last_birthday).days

    # Next birthday
    try:
        next_bday = date(today.year, birthdate.month, birthdate.day)
        if next_bday <= today:
            next_bday = date(today.year + 1, birthdate.month, birthdate.day)
    except ValueError: # Feb 29 on non-leap year
        next_bday = date(today.year + 1, birthdate.month, birthdate.day)

    days_to_next = (next_bday - today).days
    total_days = (today - birthdate).days

    return {
        'years': years,
        'days_since_last_bday': days_since,
        'total_days': total_days,
        'total_hours': total_days * 24,
        'birth_weekday': birthdate.strftime('%A'),
        'next_birthday': next_bday,
        'days_to_next_bday': days_to_next,
    }

def event_countdown(events: list[tuple]) -> None:
    """Print countdown in days to each upcoming event."""
```

```

    today = date.today()
    print('\n—— UPCOMING EVENT COUNTDOWNS ——')
    for name, ev_date in events:
        delta = (ev_date - today).days
        status = f'{delta} days away' if delta > 0 else ('TODAY!' if delta == 0
else 'Passed')
        print(f'    {name:<30} {ev_date}    [{status}]')

# — Main —————
from datetime import date

dob = date(2003, 8, 15)    # Change this to your birthdate
info = calculate_age(dob)

print('=' * 52)
print('          PERSONAL STATISTICS')
print('=' * 52)
print(f'  Date of Birth          : {dob.strftime("%d %B %Y")}')
print(f'  Born on a                : {info["birth_weekday"]}')
print(f'  Age                      : {info["years"]} years')
print(f'  Days since birthday      : {info["days_since_last_bday"]} days')
print(f'  Total days lived         : {info["total_days"]:,} days')
print(f'  Total hours lived        : {info["total_hours"]:,} hours')
print(f'  Next Birthday           : {info["next_birthday"].strftime("%d %B %Y")}')
print(f'  Days to next b'day       : {info["days_to_next_bday"]} days')

# Upcoming national events (Indian calendar)
events = [
    ('Republic Day',      date(2025, 1, 26)),
    ('Holi',              date(2025, 3, 14)),
    ('Independence Day',  date(2025, 8, 15)),
    ('Gandhi Jayanti',    date(2025, 10, 2)),
    ('Diwali',            date(2025, 10, 20)),
    ('Christmas',         date(2025, 12, 25)),
]
event_countdown(events)

```

SAMPLE OUTPUT

```

=====
          PERSONAL STATISTICS
=====
  Date of Birth          : 15 August 2003
  Born on a              : Friday
  Age                    : 21 years
  Days since birthday    : 164 days
  Total days lived       : 7,834 days
  Total hours lived      : 188,016 hours
  Next Birthday         : 15 August 2025
  Days to next b'day     : 201 days

—— UPCOMING EVENT COUNTDOWNS ——
  Republic Day          2025-01-26  [0 days away]
  Holi                  2025-03-14  [47 days away]
  Independence Day      2025-08-15  [201 days away]
  Gandhi Jayanti        2025-10-02  [249 days away]
  Diwali                2025-10-20  [267 days away]

```


5. Conclusion & Learnings

This assignment provided a thorough hands-on exploration of three important Python standard-library packages. Through practical programs executed on Google Colab, each package's functionality was demonstrated end-to-end, from basic usage to applied, real-world scenarios. The following key insights were gained:

Package	Concepts Demonstrated	Real-World Application
random	random(), randint(), choice(), shuffle(), sample(), seed(), uniform(), choices()	Games, simulations, Monte Carlo methods, lottery, statistical sampling
math	Constants (π , e), sqrt, hypot, log, exp, trig functions, ceil/floor, factorial, comb, gcd, lcm	Geometry calculation, scientific computing, engineering formulas
datetime	date, time, datetime, timedelta, strftime, strptime, timezone	Age calculators, event scheduling, log timestamps, countdowns